ARTIFICIAL INTELLIGENCE (CT 653)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 3: Search Techniques

Syllabus Hours: 5 hrs | Marks Weight: 9 marks

Complete Exam-Ready Answers | All Past Questions Covered (2070–2082)

CHAPTER 3 SYLLABUS TOPICS

- 3.1 Uninformed Search — Depth First Search (DFS), Breadth First Search (BFS), Depth Limit Search, Strategy Comparison
- 3.2 Informed Search — Hill Climbing, Best First Search, Greedy Search, A* Search
- 3.2 Adversarial Search — Minimax Procedure, Alpha-Beta Pruning

Prepared by: Santosh Kushwaha (ACE079BCT)

SECTION 1: Uninformed Search — BFS, DFS, Depth-Limited Search & Comparison

Exam Ref: 2082 Kartik, 2081 Chaitra, 2080 Ashwin Back, 2079 Jestha, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2077 Chaitra, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2073 Magh, 2072 Ashwin, 2071 Bhadra

Q1. Why is searching important in problem solving? Differentiate between Depth First Search and Breadth First Search with their performance criteria. [Most Repeated] **[2+6]**
(2082 Kartik, 2080 Ashwin Back, 2079 Chaitra, 2074 Bhadra, 2072 Ashwin)

Answer: Why Searching is Important

Searching is the fundamental mechanism by which an AI agent finds a sequence of actions that leads from the initial state to the goal state. It is important because:

1. Most AI problems have a huge number of possible states — there is no direct formula to jump to the solution.

2. The correct sequence of actions is unknown in advance; the agent must systematically explore possibilities.

3. Search underpins almost all AI tasks: planning, game playing, puzzle solving, robot navigation, theorem proving.

4. Without search, the system would need an explicit rule for every possible situation — impossible for complex domains.

Depth First Search (DFS)

Strategy: Explores as DEEP as possible along each branch before backtracking. Uses a STACK (LIFO) — or equivalently, recursion.

Algorithm:

```
1. Push start node onto STACK
2. Loop:
   a. If STACK empty → FAILURE
   b. Pop node N from STACK
   c. If N is GOAL → SUCCESS, return path
   d. Else: expand N; push children onto STACK (right to left)
   e. Repeat
```

Breadth First Search (BFS)

Strategy: Explores all nodes at the current depth level before going deeper. Uses a QUEUE (FIFO).

Algorithm:

```
1. Enqueue start node into QUEUE
2. Loop:
   a. If QUEUE empty → FAILURE
   b. Dequeue node N from front of QUEUE
   c. If N is GOAL → SUCCESS, return path
   d. Else: expand N; enqueue all children at rear
   e. Repeat
```

BFS vs DFS — Performance Criteria Comparison

Criteria	Breadth First Search (BFS)	Depth First Search (DFS)

Data Structure	Queue (FIFO)	Stack (LIFO) / Recursion
Exploration Order	Level by level (shallow first)	Deep along one branch first
Completeness	Complete — always finds solution if one exists	Incomplete — may loop in infinite branches
Optimality	Optimal — finds shallowest (fewest steps) solution (if cost=1/step)	Not optimal — may find deeper solution first
Time Complexity	O(b^d) — b=branching factor, d=depth of solution	O(b^m) — m=max depth of tree
Space Complexity	O(b^d) — must store all nodes at current level	O(b×m) — only current path + siblings
Memory Usage	Very HIGH — stores all frontier nodes	Very LOW — only stores current path
Best Use Case	When solution is shallow; when all step costs equal	When solution is deep; memory is limited
Completeness in infinite space	Yes, if branching factor is finite	No — can get trapped in infinite loop
Example	Find shortest path in unweighted graph	Maze solving, exhaustive search

Example Tree Traversal: For a tree S→{A,B}, A→{C,D}, B→{E,F}, Goal=E:

- BFS order: S, A, B, C, D, E ← found at depth 2 (optimal)
- DFS order: S, A, C, D, B, E ← found (but explored unnecessary branch first)

Q2. What is the advantage of depth limit search? Compare it with other search strategies. **[4+4]** (2079 Ashwin Back, 2071 Bhadra)

Answer: Depth-Limited Search (DLS)

Problem with DFS: DFS can get trapped in infinite loops (infinite state spaces) or very deep branches, never finding the solution.

Depth-Limited Search (DLS): DFS with a predetermined depth limit L. The search never goes deeper than L levels. Nodes at depth L are treated as if they have no children.

Algorithm:

```
DLS(node, depth_limit):
  if node is GOAL → return SUCCESS
  if depth_limit == 0 → return CUTOFF
  for each child of node:
    result = DLS(child, depth_limit - 1)
    if result ≠ CUTOFF → return result
  return CUTOFF
```

Advantage over DFS: Avoids infinite loops and excessively deep branches. Saves memory like DFS. Useful when we know the maximum depth of the solution.

Disadvantage: If L is set too small, the solution may not be found (incompleteness). If L is too large, it wastes time exploring deep paths.

Iterative Deepening Search (IDS): Solves the depth-limit problem by running DLS with L=0,1,2,3,… until solution is found. Combines BFS's completeness/optimality with DFS's low memory.

Full Comparison of Uninformed Search Strategies

Strategy	Complete?	Optimal?	Time	Space	Notes
BFS	Yes	Yes (uniform cost)	O(b^d)	O(b^d)	Best for shallow solutions
DFS	No (infinite)	No	O(b^m)	O(b·m)	Good memory, may not terminate
DLS	No (if L<d)	No	O(b^L)	O(b·L)	Needs good depth estimate
IDS	Yes	Yes (uniform cost)	O(b^d)	O(b·d)	Best uninformed strategy overall
Uniform Cost	Yes	Yes (min cost)	O(b^(C*/ε))	O(b^(C*/ε))	Finds cheapest solution
Bidirectional	Yes	Yes	O(b^(d/2))	O(b^(d/2))	Searches from both ends

SECTION 2: Informed Search — Hill Climbing, Greedy Best-First, A* Algorithm

Exam Ref: 2081 Chaitra, 2080 Chaitra, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra

Q3. Why is searching important in problem solving? What are the drawbacks of Greedy Best-First Search and how does A* search solve it? Explain with example. [Most Repeated] **[2+7]** (2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2074 Bhadra)

Answer: Informed (Heuristic) Search

Informed Search: Uses problem-specific knowledge (a HEURISTIC function h(n)) to guide the search toward the goal more efficiently than uninformed search.

Heuristic Function h(n): Estimates the cost from node n to the goal. It is an educated guess — it doesn't have to be exact, but a good heuristic greatly reduces search effort.

Example heuristics: h(n) = straight-line distance to goal (for map navigation); h(n) = number of misplaced tiles (for 8-puzzle).

Greedy Best-First Search

Strategy: Always expand the node with the SMALLEST h(n) — the node that APPEARS closest to the goal according to the heuristic. Ignores path cost g(n).

Evaluation function: f(n) = h(n)

Drawbacks of Greedy Best-First Search:

1. NOT OPTIMAL: It finds A solution, but not necessarily the SHORTEST/CHEAPEST one. It follows what looks best locally but can take a longer overall path.

2. NOT COMPLETE: Can get trapped in loops if no cycle detection is used.

3. Can be misled by a bad heuristic: If h(n) is inaccurate, greedy search can head in completely the wrong direction.

4. Ignores actual path cost: A path that looks close to goal (small h) may have been very expensive to reach (large g).

A* Search Algorithm

Strategy: Expands the node with the smallest f(n) = g(n) + h(n), where:

- g(n) = actual cost from start to node n (backward looking)
- h(n) = estimated cost from n to goal (forward looking, heuristic)
- f(n) = estimated total cost of cheapest path through n

Key Properties of A*:

- COMPLETE: Always finds a solution if one exists (with finite branching factor).
- OPTIMAL: Finds the CHEAPEST solution — guaranteed if h(n) is ADMISSIBLE.

Admissible Heuristic: h(n) NEVER OVERESTIMATES the true cost to reach the goal. h(n) ≤ h*(n) where h*(n) is the true cost. Example: straight-line distance never overestimates actual road distance.

A* Algorithm:

```
1. Put start node in OPEN list with f(start) = g(start) + h(start)
2. Loop:
   a. If OPEN is empty → FAILURE
   b. Pick node N with LOWEST f(n) from OPEN
   c. Move N to CLOSED list
```

```
d. If N is GOAL → return path (trace back via parent pointers)
e. For each neighbor M of N:
   - Compute g(M) = g(N) + cost(N,M)
   - Compute f(M) = g(M) + h(M)
   - If M not in OPEN or CLOSED → add to OPEN
   - If M in OPEN with higher f → update
   - If M in CLOSED with higher f → re-open
f. Repeat
```

Worked Example: A* on Graph (2081 Chaitra exam graph)

Graph: S→A(6), S→B(5), S→C(10), A→E(6), A→B(6 via back), B→D(7), B→E(6→connected via A), C→D(6), D→F(6), E→F(4), F→G(3)

Heuristic values (h): S=17, A=10, B=13, C=5, D=4, E=2, F=1, G=0

Step	Node Expanded	g(n)	h(n)	f(n)	Action / Notes
1	S (start)	0	17	17	Expand S: generate A(f=0+6+10=16), B(f=0+5+13=18), C(f=0+10+5=15)
2	C	10	5	15	Lowest f=15. Expand C: generate D(g=10+6=16, f=16+4=20)
3	A	6	10	16	Next lowest f=16. Expand A: generate E(g=6+6=12, f=12+2=14)
4	E	12	2	14	Lowest f=14. Expand E: generate F(g=12+4=16, f=16+1=17)
5	B	5	13	18	Check B: already in OPEN but update path if cheaper
6	F	16	1	17	Expand F: generate G(g=16+3=19, f=19+0=19)
7	G	19	0	19	GOAL REACHED!

Optimal Path: S → A → E → F → G with total cost = 6+6+4+3 = 19

Greedy path would have taken: S → C → D → F → G = 10+6+6+3 = 25 (not optimal — A* does better)

Q4. Explain A* algorithm. Use the following figure where starting node is S and destination is G. (Note: values in nodes = heuristic cost to goal; values on edges = step cost.) **[8]** (2081 Chaitra, 2080 Chaitra, 2079 Chaitra)

Answer: Full A* Trace (Generic Graph S→G)

For the standard exam graph (S→A=6, S→B=5, S→C=10, A→E=6, B→D=7, C→D=6, D→F=6, E→F=4, F→G=3):

Heuristic: S=17, A=10, B=13, C=5, D=4, E=2, F=1, G=0

OPEN list (sorted by f) | CLOSED list

```
Initial: OPEN={S(f=17)}, CLOSED={}

Step 1: Expand S (f=17)
```

```
  CLOSED={S}
  Children: A(g=6,h=10,f=16), B(g=5,h=13,f=18), C(g=10,h=5,f=15)
  OPEN={C(15), A(16), B(18)}

Step 2: Expand C (f=15, lowest)
  CLOSED={S,C}
  Children: D(g=10+6=16, h=4, f=20)
  OPEN={A(16), B(18), D(20)}

Step 3: Expand A (f=16)
  CLOSED={S,C,A}
  Children: E(g=6+6=12, h=2, f=14)
  OPEN={E(14), B(18), D(20)}

Step 4: Expand E (f=14, lowest)
  CLOSED={S,C,A,E}
  Children: F(g=12+4=16, h=1, f=17)
  OPEN={F(17), B(18), D(20)}

Step 5: Expand F (f=17)
  CLOSED={S,C,A,E,F}
  Children: G(g=16+3=19, h=0, f=19)
  OPEN={B(18), D(20), G(19)}

Step 6: Expand B (f=18)
  CLOSED={S,C,A,E,F,B}
  Children: D already in OPEN with f=20; g via B=5+7=12 < 16 ✓ UPDATE
  D updated: g=12, h=4, f=16
  OPEN={D(16), G(19)}

Step 7: Expand D (f=16)
  Children: F already in CLOSED — skip (or check if cheaper path)
  OPEN={G(19)}

Step 8: Expand G (f=19) → GOAL! ✓

OPTIMAL PATH: S→A→E→F→G
Total cost: 6 + 6 + 4 + 3 = 19
```

A* vs Greedy Comparison for same graph:

Measure	A* (f=g+h)	Greedy Best-First (f=h only)
Path found	S→A→E→F→G (cost=19)	S→C→D→F→G (cost=25)
Is it optimal?	YES ✓	NO ✗
Nodes expanded	6-8 nodes	4-5 nodes (but wrong answer)
Completeness	Yes	Yes (with visited check)

Q5. Explain A* algorithm. Discuss about Hill Climbing Search and the problems associated with it. [2077 Chaitra, 2075 Bhadra, 2073 Bhadra] **[2+7]** (2077 Chaitra)

Answer: Hill Climbing Search

Hill Climbing: A local search algorithm that starts at the current state and always moves to the NEIGHBOR with the highest value (or lowest cost). It does NOT maintain a search tree — only the current state is remembered.

Analogy: Like climbing a hill in dense fog — you can only see the immediate slope around you. You always step in the uphill direction.

Algorithm:

```
current = initial_state
loop:
  neighbors = generate_neighbors(current)
  best = highest-value neighbor
  if value(best) ≤ value(current):
    return current   ← local maximum reached
  current = best
```

Variants: Simple Hill Climbing (first better neighbor), Steepest Ascent (best neighbor), Stochastic (random better neighbor).

Problems with Hill Climbing:

Problem	Description	Visual Analogy	Solution
Local Maximum	Reached a state better than all neighbors, but NOT the global optimal	Stuck on a small hill, not the tallest mountain	Random restart, simulated annealing
Plateau	All neighbors have equal value — no direction to move	Flat hilltop or shoulder	Allow sideways moves (up to a limit)
Ridge	Narrow peak running diagonally — moves in any single direction go downhill	A knife-edge ridge	Move in multiple directions simultaneously
Infinite Loop	On a plateau, sideways moves may cycle forever	Walking in circles on flat ground	Limit number of sideways moves

Simulated Annealing: Solves the local maximum problem by occasionally accepting WORSE moves with a probability that decreases over time (like cooling metal). Early on (high temperature) — accepts bad moves freely. Later (low temperature) — only accepts improvements.

Random Restart Hill Climbing: Run hill climbing multiple times from randomly chosen initial states. Take the best result. With enough restarts, finds the global optimum.

SECTION 3: Adversarial Search — Minimax & Alpha-Beta Pruning

Exam Ref: 2082 Kartik, 2081 Ashwin, 2080 Ashwin Back, 2079 Chaitra, 2079 Jestha, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2071 Magh, 2070 Bhadra

Q6. What are the drawbacks of the Min-Max algorithm and explain how the Alpha-Beta pruning algorithm solves it with a suitable example. [Most Repeated] **[3+6]** (2082 Kartik, 2081 Ashwin, 2080 Ashwin Back, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh)

Answer: Minimax Algorithm

Game Playing in AI: Two-player, zero-sum, perfect-information games (Chess, Tic-Tac-Toe, Go). One player tries to MAXIMIZE score (MAX player), the other tries to MINIMIZE it (MIN player).

Minimax Strategy: Assume both players play OPTIMALLY. MAX chooses the move that leads to the maximum minimax value; MIN chooses the move that leads to the minimum minimax value.

Algorithm (recursive):

```
MINIMAX(node, depth, isMaximizing):
  if depth==0 or node is terminal:
      return evaluate(node)   ← static evaluation
  if isMaximizing:
      best = -∞
      for each child of node:
          val = MINIMAX(child, depth-1, FALSE)
          best = max(best, val)
      return best
  else:  ← MIN player
      best = +∞
      for each child of node:
          val = MINIMAX(child, depth-1, TRUE)
          best = min(best, val)
      return best
```

Drawbacks of Minimax:

1. EXPONENTIAL TIME COMPLEXITY: O(b^m) where b=branching factor, m=depth. Chess: b≈35, m≈100 → 35^100 nodes — completely infeasible.

2. Evaluates ALL leaf nodes: Explores branches that cannot possibly affect the final decision — wasted computation.

3. Memory intensive: Must store the entire game tree (or at least the current path).

4. Not practical for deep trees without pruning.

Alpha-Beta Pruning

Key Idea: Alpha-Beta pruning eliminates branches of the game tree that CANNOT possibly affect the minimax decision. It maintains two values:

- α (alpha): The BEST value that MAX can guarantee so far (MAX's lower bound). Initially α = −∞
- β (beta): The BEST value that MIN can guarantee so far (MIN's upper bound). Initially β = +∞

Pruning Conditions:

- At a MIN node: if the current value ≤ α → PRUNE (MIN will never choose this, MAX already has better)

- At a MAX node: if the current value ≥ β → PRUNE (MAX will never choose this, MIN already has better)

Algorithm:

```
ALPHA_BETA(node, depth, α, β, isMax):
  if depth==0 or terminal: return evaluate(node)
  if isMax:
      for each child:
          val = ALPHA_BETA(child, d-1, α, β, FALSE)
          α = max(α, val)
          if α ≥ β: break  ← β-cutoff (prune)
      return α
  else:
      for each child:
          val = ALPHA_BETA(child, d-1, α, β, TRUE)
          β = min(β, val)
          if β ≤ α: break  ← α-cutoff (prune)
      return β
```

Worked Example: Alpha-Beta on a Game Tree

Tree (MAX at root, MIN at depth 1, leaf values at depth 2):

```
          MAX
         /   \
       MIN   MIN
      /   \ /   \
     3   5 2    9
```

Pure Minimax:

```
Left MIN node:  min(3,5) = 3
Right MIN node: min(2,9) = 2
ROOT MAX node:  max(3,2) = 3
All 4 leaf nodes evaluated.
```

With Alpha-Beta Pruning (α=-∞, β=+∞ at root):

```
Step 1: Visit left MIN node (α=-∞, β=+∞)
  Evaluate leaf 3: β = min(+∞, 3) = 3. α still -∞. 3 not ≤ -∞, continue.
  Evaluate leaf 5: β = min(3, 5) = 3. Return 3 to MAX.
  MAX root: α = max(-∞, 3) = 3.

Step 2: Visit right MIN node (α=3, β=+∞)
  Evaluate leaf 2: β = min(+∞, 2) = 2.
  Check: β(2) ≤ α(3)? YES → α-cutoff! PRUNE leaf 9.
  Return 2 to MAX (but MAX won't choose 2 < α=3 anyway).

Result: Root value = 3. Only 3 leaves evaluated (leaf 9 pruned). ✓
```

Bigger example (from 2076 Baisakh exam tree with leaves 3,5,10 | 2,8,19 | 2,7,3):

```
MAX (root)
```

```
          /       |       \
        MIN       MIN      MIN
       /|\       /|\      /|\
      3 5 10   2 8 19   2 7  3

Left MIN:  min(3,5,10)=3.  ROOT α=3.
Middle MIN (α=3, β=+∞):
  Leaf 2: β=2. β(2)≤α(3)? YES → PRUNE leaves 8,19.
  Return 2. MAX ignores (2<α=3).
Right MIN (α=3, β=+∞):
  Leaf 2: β=2. β(2)≤α(3)? YES → PRUNE leaves 7,3.
  Return 2. MAX ignores.

FINAL ROOT VALUE = 3 (from left MIN branch)
Leaves evaluated: 3,5,10,2,2 = 5 out of 9. (4 pruned!)
```

Alpha-Beta Pruning Benefits:

Aspect	Minimax	Alpha-Beta Pruning
Nodes evaluated	O(b^m)	O(b^(m/2)) best case
Same result?	Yes — always gives same answer as minimax	YES — identical result, guaranteed
Effective branching factor	b	√b (effectively)
For Chess (b=35,m=8)	35^8 ≈ 2.2 billion nodes	35^4 ≈ 1.5 million nodes (1000× faster)
Pruning amount	None	Up to 50% of nodes in random order; up to 75% with good move ordering

---

Q7. What is minmax search? Apply minmax search along with alpha-beta pruning for the following tree at max node at top. Leaf values: [3,9,2,7,2,5,0,2,5,8,3,14]. [2079 Chaitra, 2075 Bhadra] **[2+6]** (2079 Chaitra)

Answer: Minimax + Alpha-Beta on Exam Tree

Tree structure (from 2079 Chaitra exam): MAX at root, MIN at depth 1, MAX at depth 2, leaf nodes at depth 3.

Leaf values (left to right): 3, 9, 2, 7, 2, 5, 0, 2, 5, 8, 3, 14

```
                    MAX (root)
              /               \
            MIN               MIN
          /     \           /     \
        MAX    MAX         MAX    MAX
       / \    / \         / \    / \
      3  9   2  7       2  5   0  2   5  8  3  14
```

(Adjust grouping based on actual tree branching in exam)

Minimax Calculation (bottom-up):

```
Depth-2 MAX nodes:
  MAX(3,9)=9,  MAX(2,7)=7,  MAX(2,5)=5,  MAX(0,2)=2
  MAX(5,8)=8,  MAX(3,14)=14

Depth-1 MIN nodes:
  MIN(9,7)=7,  MIN(5,2)=2,  MIN(8,14)=8

Root MAX: MAX(7,2,8) = 8
MINIMAX VALUE = 8
```

Alpha-Beta Pruning Trace:

```
ROOT MAX: α=-∞, β=+∞

Left MIN subtree (α=-∞, β=+∞):
  Left MAX (α=-∞, β=+∞):
    Leaf 3: α=max(-∞,3)=3
    Leaf 9: α=max(3,9)=9. Return 9 to MIN.
  MIN: β=min(+∞,9)=9. Continue.
  Right MAX (α=-∞, β=9):
    Leaf 2: α=2
    Leaf 7: α=max(2,7)=7. α(7)<β(9), continue.
    Return 7 to MIN.
  MIN: β=min(9,7)=7. Return 7 to ROOT.
ROOT: α=max(-∞,7)=7.

Middle MIN subtree (α=7, β=+∞):
  Left MAX (α=7, β=+∞):
    Leaf 2: α=max(7,2) — wait, α stays at parent value.
    At MAX node: pass α=7, β=+∞ down.
    Leaf 2: local α stays 7 (2<7, so no update).
    Actually local best=2. Return 2? No: MAX picks max(leaves)
    max(2,5)=5. Return 5 to MIN.
  MIN: β=min(+∞,5)=5. β(5)≤α(7)? YES → PRUNE right MAX subtree!
  (Leaves 0,2 are PRUNED.)
  Return 5 to ROOT. ROOT α=max(7,5)=7 (no update).

Right MIN subtree (α=7, β=+∞):
  Left MAX (α=7, β=+∞):
    max(5,8)=8. Return 8 to MIN.
  MIN: β=min(+∞,8)=8. β(8)>α(7), continue.
  Right MAX (α=7, β=8):
    Leaf 3: local max starts at 3. α still 7.
    Leaf 14: local max=14. 14≥β(8) → PRUNE (β-cutoff). Return 14.
    Actually: at MAX node if val≥β, prune. 14≥8 → prune.
  MIN: β=min(8,14)→but we pruned. MIN returns 8.
ROOT: α=max(7,8)=8.

FINAL ANSWER: Root value = 8
Pruned nodes: {0,2} in middle subtree + early exit in right subtree
```

Q8. List down the approaches for evaluating searching methods. What are the drawbacks of Min-Max algorithm and explain how Alpha-Beta pruning algorithm solves it with a suitable example. [2+6] (2081 Ashwin)

Answer: Approaches for Evaluating Search Algorithms

A search algorithm is evaluated using four performance criteria:

Criterion	Definition	BFS	DFS	A*	Alpha-Beta
Completeness	Does it always find a solution if one exists?	Yes	No (infinite)	Yes	Yes
Optimality	Does it always find the BEST (lowest cost) solution?	Yes (unit cost)	No	Yes (admissible h)	Yes (optimal play)
Time Complexity	How many nodes are generated/expanded?	O(b^d)	O(b^m)	O(b^d) best	O(b^(m/2))
Space Complexity	How much memory is used?	O(b^d)	O(b·m)	O(b^d)	O(b·m)

b=branching factor, d=depth of optimal solution, m=max depth of tree

SECTION 4: Additional Questions — Hill Climbing Details & Strategy Selection

Exam Ref: 2078 Chaitra, 2077 Chaitra, 2075 Bhadra, 2074 Bhadra, 2072 Ashwin

Q9. Discuss the evaluation criteria for search algorithm. State the problems in hill climbing search algorithm and how A* overcomes the limitations of Greedy Best-First Search. **[4+4]** (2075 Bhadra, 2074 Bhadra)

Answer: How A* Overcomes Greedy Limitations

Feature	Greedy Best-First Search	A* Search
Evaluation function	f(n) = h(n) only	f(n) = g(n) + h(n)
Considers path cost?	NO — ignores how expensive it was to reach n	YES — g(n) tracks actual cost so far
Optimality	Not optimal — may find suboptimal path	Optimal — finds cheapest path (if h admissible)
Completeness	Incomplete (can loop)	Complete
Behavior	Rushes toward what looks close to goal	Balances closeness to goal AND cost so far
Risk of being misled	HIGH — bad heuristic leads it astray	LOWER — actual cost g(n) corrects heuristic bias
Example failure	Takes expensive shortcut that looks close	Would not take expensive shortcut

Summary: Greedy search is like a runner who always sprints toward where the finish looks closest, ignoring hills and detours. A* is like a runner who considers both the distance already run AND the estimated remaining distance — finding the truly shortest total route.

Q10. Explain BFS and DFS with suitable example. Why simulated annealing is important? [2079 Jestha, 2078 Chaitra] **[2+6]** (2079 Jestha)

Answer: BFS vs DFS — Worked Example

Consider graph: S→{A,B}, A→{C,D}, B→{E,F}, C→{G}, Goal = G

Algorithm	Traversal Order	Data Structure	Found at
BFS	S → A → B → C → D → E → F → G	Queue (FIFO)	Depth 3 (optimal — shortest path)
DFS	S → A → C → G	Stack (LIFO)	Depth 3 (coincidentally optimal here)
DFS (bad case)	S → A → C → (infinite loop)	Stack	Never, if graph has cycle and no visited check

Simulated Annealing

Why it is important: Hill climbing gets stuck at LOCAL maxima — states better than all neighbors but not globally optimal. Simulated Annealing (SA) escapes local maxima by occasionally accepting WORSE moves.

Inspired by: The metallurgical annealing process — heating metal and cooling it slowly to minimize defects (energy). High temperature → more disorder/random. Low temperature → settles into optimal structure.

Algorithm:

```
current = initial_state
T = initial_temperature (high)
loop:
  next = random_neighbor(current)
  ΔE = value(next) - value(current)
  if ΔE > 0: current = next  ← always accept improvement
  else: accept next with probability e^(ΔE/T)  ← sometimes accept worse
  T = T × cooling_rate  ← decrease temperature
  if T ≈ 0: return current
```

Key insight: At high T, e^(ΔE/T) ≈ 1 — almost always accept. At low T, e^(ΔE/T) ≈ 0 — rarely accept. This allows escaping local maxima early, then converging to optimum.

Importance: SA is provably optimal — given a slow enough cooling schedule, it will find the global optimum. Used in VLSI circuit design, scheduling, TSP, protein folding.

QUICK REVISION: Chapter 3 — Key Points Summary

Topic	Key Points to Remember	Expected Marks
BFS vs DFS	BFS: Queue, complete, optimal, O(b^d) space. DFS: Stack, incomplete, not optimal, O(bm) space	4-6
Depth-Limited & IDS	DLS adds depth limit L to DFS. IDS tries L=0,1,2,…; complete + optimal + low memory	3-4
Search Evaluation	4 criteria: Completeness, Optimality, Time O(), Space O()	2-3
Hill Climbing	Always move to best neighbor; problems: local max, plateau, ridge; fix: random restart, SA	4-6
Greedy Best-First	f(n)=h(n); fast but not optimal, not complete	2-3
A* Algorithm	f(n)=g(n)+h(n); complete + optimal (if h admissible); OPEN/CLOSED list approach	6-8
Admissible Heuristic	h(n) ≤ h*(n) — never overestimates. Required for A* optimality	1-2
Minimax	MAX picks max, MIN picks min; O(b^m) time; optimal but infeasible for deep trees	4-5
Alpha-Beta Pruning	Prune when α≥β; same result as minimax; O(b^(m/2)) time; α=MAX's best, β=MIN's best	5-8
Simulated Annealing	Accept worse moves with probability e^(ΔE/T); escape local maxima; temperature cools	3-4

Best of Luck! | TU IOE BCT | CT 653 AI | Chapter 3: Search Techniques